

**Comparative Reward Engineering for Specialized Reinforcement Learning Agents in
Rocket League**

By: Samiul Saimon

Introduction

Reinforcement learning (RL) frames decision making as an interaction loop in which an agent observes the state of an environment, selects an action, and receives a scalar reward (or penalty) that quantifies how good that action was. Over many such interactions, the agent learns a policy that maximizes the cumulative reward it expects to receive. While the math behind RL works for almost anything, getting an agent to behave correctly in practice is incredibly difficult because it is so sensitive to the rewards we give it. In other words, the actual behavior we get depends entirely on how we design the reward. Rewards that look reasonable on paper can produce agents that behave nothing like the designer intended, and a lot of applied RL work involves carefully shaping rewards so that they incentivize the desired behavior without accidentally rewarding shortcuts that the designer never anticipated (reward hacking).

This project investigates that tradeoff in a complex physics simulation of Rocket League, a video game in which two teams of cars compete to drive a large ball into each other's nets. The scope is narrowed to a specialized defensive role, often called the "goalie" role by players. The objective is not to produce a human-competitive bot but to compare how three families of reward functions, trained under otherwise identical conditions, shape the behavior of a Proximal Policy Optimization (PPO) agent over forty million simulation time steps each: a sparse event-based reward, a continuous physics-based dense reward, and a context-aware hybrid reward. The context-aware design also serves as an iterative correction case study because its initial form produced a classic example of reward hacking. The deliverables include a reproducible training pipeline built on RLGym v2 and RocketSim, a custom Weights & Biases (W&B) metrics logger for domain-specific goalie metrics, five saved policy checkpoints, short visual playbacks, and the comparative analysis presented in this report.

Background

This project uses Proximal Policy Optimization (PPO), a policy-gradient reinforcement learning method. Unlike value-based approaches like Deep Q-Learning - which estimate scores for every action and act greedily - PPO directly trains a neural network to output actions, continuously tuning its weights to maximize expected cumulative rewards. The "proximal" part refers to a safety feature: a mathematical clipping rule that limits how far a new strategy can drift from the old one during an update step. This prevents reckless behavioral shifts, yielding more stable training in high-dimensional, chaotic environments like Rocket League.

PPO implements this via an actor-critic architecture, splitting the workload between two cooperating neural networks that share the same observations. The Actor (the policy) outputs a probability distribution over possible actions to choose the car's next move. The Critic (the value function) estimates the total future reward expected from the current state. Rather than controlling the car, the Critic acts as an internal judge; by providing an immediate baseline of state quality, it reduces variance and removes chaotic guesswork from the learning process, making training faster and more stable.

To track how well PPO is learning, we monitor three key metrics. First, KL divergence measures how different the new policy (strategy) is from the old policy after an update. Low KL means the policy is barely changing; very high or spiky KL means the agent is making large, potentially unstable changes. Second, clip fraction measures how often the surrogate objective hit its safety cap during minibatch updates, in other words, how often PPO had to "rein in" an update because it was too aggressive. Clip fractions near zero mean the trust region is rarely binding; clip fractions in the 0.10 to 0.20 range mean the policy consistently wants to move farther than PPO allows. Finally, policy entropy measures how stochastic (random) the policy

currently is. High entropy means the action distribution is still spread out and the agent is exploring; entropy that drops too quickly can indicate premature convergence to a narrow strategy before fully exploring its options.

Reward engineering requires balancing sparse and dense signals. A sparse reward outputs zero unless a major event occurs, such as a goal or a save. While perfectly aligned with the true objective, sparse signals are incredibly difficult to learn from initially, as a random policy may rarely stumble into a positive example. Conversely, a dense reward provides continuous feedback at every timestep using proxy metrics like distance or velocity. Dense rewards accelerate learning but risk misalignment, as agents often exploit these heuristics to farm points without achieving the actual goal. This project also evaluates a context-aware hybrid reward that attempts to combine a continuous dense baseline with specific defensive rules.

Finally, the training setup utilizes self-play, where both teams are controlled by the same evolving policy. This eliminates the need to hardcode a fixed opponent, but it causes environment difficulty to increase in lockstep with agent competence. Because both sides move randomly early in training, what is intended as a goalie task initially resembles two confused cars flailing near a stationary ball - a dynamic that plays a part in shaping the experimental results.

Environment and Setup

The simulation environment is RLGym v2 with the RocketSim physics backend. RLGym is the framework that exposes Rocket League as a standard Gym-style environment, providing state mutators, reward functions, observation builders, action parsers, and termination conditions. RocketSim is a lightweight C++ physics engine that replicates the game's mechanics without the graphic rendering overhead, making 40-million-timestep runs computationally feasible. The PPO

implementation comes from the `rlgym-ppo` library, which provides parallelized rollout workers, checkpointing, and native Weights & Biases (W&B) logging.

Training was conducted on a Windows 11 machine featuring an Intel i5-11500 CPU (6 cores/12 threads) and an NVIDIA RTX 3060 GPU. This hardware distribution aligns with the training architecture: RocketSim runs on the CPU via eight parallel worker processes, while the PPO policy and critic updates execute on the GPU via CUDA-enabled PyTorch. This pipeline keeps the GPU busy processing continuous experience batches generated by the background CPU threads.

The software stack is managed via Miniconda with Python 3.9 and PyTorch mapped to CUDA 12.1. The RLGym v2 stack relies on three primary packages: `rlgym`, `rlgym-rocket-league` (which includes standard soccer collision meshes, eliminating the need for asset dumping), and `rocketsim`. The `rlgym-ppo` library was installed directly from source, alongside the official wandb client. Visual playback is handled by RLViser (v0.8.2), an external Rust binary triggered via a `--render` flag; this specific version was selected to prevent memory allocation failures observed in newer builds.

The project code itself is structured into two main components. The file `trainer.py` defines the environment builder, the custom W&B metrics logger class `GoalieWandbMetricsLogger`, the command-line interface for selecting a reward strategy and run name, and the PPO learner configuration. The file `rewards.py` contains the three reward functions: `GoalieSparseReward`, `PhysicsDenseReward`, and `ContextAwareReward`. Checkpoints are stored locally under the `agents/<run_name>/` directory in numeric subfolders corresponding to training timesteps, while W&B metrics are mirrored locally under `wandb/` to ensure offline data accessibility.

Agents and Methods

To ensure efficient training, the project relies on a streamlined action and observation pipeline. The agent uses a discrete action parser selecting from a lookup table of standard Rocket League control combinations. This setup is wrapped in a RepeatAction function that holds each selected move for eight simulation ticks, accelerating training at the cost of slightly coarser control. Observations are managed via RLGym's DefaultObs, which normalizes all positional, velocity, and angular data into a standardized $[-1, 1]$ range to ensure efficient neural network processing.

Both the policy (Actor) and value (Critic) networks are three-layer multilayer perceptrons with 256 hidden units per layer, matching standard rlgympo defaults. PPO updates execute every 50,000 collected timesteps for 10 epochs, processing the entire rollout batch at once with a default clip range of 0.2 and a learning rate of $3e-4$. Network capacity does not bottleneck the agent; performance is driven entirely by reward design and game physics.

Episodes feature symmetric 1v1 matchups starting from a standard center kickoff. Every episode terminates via a goal (GoalCondition) or a 15-second timeout (TimeoutCondition). This rapid reset structure forces the agent to spend nearly all its training time in short kickoff plays rather than long, drawn-out defensive stands. Both cars share an identical policy via symmetric self-play. While the reward function is identical, each car evaluates it relative to its own net; a defensive clear for blue yields zero reward for orange. This self-play loop causes both sides to improve in lockstep, though it pushes the agents to minimize risk, leading to passive behaviors.

The three reward functions used in this project are all implemented in rewards.py and selected through the --strategy command-line argument. The sparse reward, GoalieSparseReward, returns +1 whenever the car touches the ball on its defending half (Y-axis

> 1500 units past the centerline) and zero otherwise. Because a random policy rarely stumbles into a defensive touch early on, it provides almost no early learning signal and acts as a control condition. The dense reward, `PhysicsDenseReward`, calculates continuous values at every timestep by measuring the alignment between the ball's velocity vector and the vector pointing from the agent's goal to the ball, divided by maximum ball speed. It awards positive points when the ball moves away from the net and negative points when it travels toward it. However, this signal is entirely agnostic to who hit the ball, forcing the agent to learn causality through trial and error. The context-aware reward, `ContextAwareReward`, builds on the dense reward by adding three context terms: a facing bonus, a small boost-availability bonus (>20% boost), and a soft penalty for drifting beyond a 2200-unit "goalie radius" from the net.

The context reward was evaluated across three iterative versions to study and correct reward hacking: The initial version, `context_40m`, applied heavy weights to the context terms (facing: 0.10, boost: 0.02, position penalty: 0.10). This triggered severe reward hacking: the agent simply camped near its net, gathering boost and facing the ball to harvest constant positive rewards, rarely ever making a play. The corrected variant, `context_v2`, reduced context weights (facing: 0.05, boost: 0.005, position penalty: 0.05) and introduced two passivity penalties: a 0.03 maximum penalty that ramps up over 5 seconds of no-touch behavior, and a center-unchallenged penalty that punishes the agent for abandoning a ball near the centerline. The third and final variant, `context_v3`, maintained the v2 architecture but added a +0.30 event-based touch bonus to provide a strong positive incentive for physical contact, offsetting v2's penalty-heavy design. The reasoning was that v2 had only penalties on the context side, so the agent was learning what not to do but never receiving a strong positive pull toward actually striking the ball.

Training Procedure

Training each run uses the same command structure. For example, the dense baseline is launched with: “python trainer.py --strategy dense --run-name dense_40m --n-proc 8 --device cuda --timestep-limit 40000000”. The trainer parses these arguments, constructs an environment-builder function (using `functools.partial` for Windows multiprocessing compatibility), instantiates the `rlgym-ppo Learner`, and invokes `learner.learn()`. The learner spawns 8 parallel worker processes, each running a copy of the environment to continuously gather gameplay data using the most recent strategy. Once enough experience is collected, the learner performs a PPO update on the GPU and broadcasts the new policy back to the workers simultaneously. Checkpoints are automatically saved every 300,000 timesteps into the `agents/<run_name>/` directory. Each checkpoint folder contains the policy weights, optimizer states, tracking data JSON, and running normalization statistics for observations and rewards.

On the training hardware, a 40-million-timestep run takes roughly two hours of real-world time. This duration is consumed mostly by the CPU processing required to step the eight parallel `RocketSim` environments, rather than by the GPU updates themselves. The learner uses `standardize_returns=True` and `standardize_obs=True` to normalize returns and incoming game data using running statistics. Both settings are library defaults left unchanged to keep the baseline comparisons clean, as the project isolates the impact of different reward designs rather than fine-tuning PPO hyperparameters.

Visual playback uses the same `trainer.py` script with three extra flags: `--checkpoint-load-folder` to point at an existing run directory, `--render` to enable `RLViser`, and a `--timestep-limit` set just above the current checkpoint's timestep count so that the run continues for a few more iterations and the environment keeps stepping. Playback is therefore not a

recording of historical frames but a live rollout of the trained policy starting from the loaded checkpoint, which makes it an accurate representation of what the policy actually does at that point in training. The render flag also requires rlviser.exe to be present in the working directory, otherwise the launcher fails with a clear error message added in this project.

Results

The five runs used in the comparison are summarized in the table below. All five share identical PPO and environment configuration; only the reward function (and within the context family, the version of the context reward) differs.

Reward Type	Description
Sparse	only reward a defensive touch on agent's own half to encourage staying in the goal area.
Dense	based on how much the ball's velocity points away from the agent's own goal (encourages clearing the ball out of the goal area)
Context-Aware	dense physics signal plus positioning, orientation towards ball, and boost context (for better reactions).
Context-Aware v2	V1 + Penalty for drifting far from goal, prolonged no touch behavior, center ball unchallenged (stop reward hacking).
Context-Aware v3	V2 + event-based positive reward on any ball contact (encourage hitting ball).

Figure 1 shows the mean KL divergence across the five runs over training. KL divergence is a measure of how different the post-update policy is from the pre-update policy, averaged across the rollout batch. The sparse run drops to essentially zero KL almost immediately and stays there for the rest of training, which is consistent with a policy that has no

meaningful signal to learn from and therefore has nothing to update toward. The dense run shows the classic healthy pattern: an initial period of larger updates during early learning, followed by a steady decay toward roughly 0.003 by the end of training, indicating a policy that has been refined into a stable strategy. The three context runs all show a steady upward trend in KL throughout training, with context_v2 the most aggressive (highest KL) and context_v3 slightly more constrained than v2. They are not collapsing or diverging, but they are clearly pushing on the trust region throughout the run.

Figure 2 shows the SB3 clip fraction, which records the fraction of PPO minibatch updates in which the clipped surrogate objective actually had to be clipped. This is effectively a "how often is the safety brake being used" diagnostic. The dense run spikes briefly around the two-million-timestep mark when initial learning is most rapid, then settles to a small steady value around 0.03, meaning the trust region is rarely binding. The sparse run sits effectively at zero for the entire run. Context_40m sits between 0.08 and 0.09. Context_v2 and v3 sit between 0.12 and 0.14 for most of the training, with context_v2 slightly higher than v3. A clip fraction above 0.1 means that more than one in ten minibatch updates wanted to move farther than PPO would allow, which is consistent with reward signals that produce sharper gradients than the dense run. It is not by itself catastrophic, but it does mean the context runs are running closer to the edge of the trust region than the dense run.

Figure 3 plots the policy reward (the average per-step reward under the current policy, smoothed across iterations) over training for all five runs. The sparse run sits effectively at zero for the entire forty million steps, confirming what was already visible in the KL and clip diagnostics: a randomly initialized policy in this MDP almost never produces a defensive-half touch, so the sparse reward never fires and the policy has nothing to learn from. The dense run

shows the textbook upward climb expected of a healthy PPO learner, rising from roughly 0.35 early in training to roughly 1.66 by the end. The context_40m run climbs to approximately twenty-five and stays there. The two corrected context runs sit below the dense curve in scalar terms, with context_v2 ending near 0.95 and context_v3 ending near 1.28, and v3 is consistently slightly above v2 in the second half of training. The policy reward chart is not a fair scoreboard across different reward designs, the behavior metrics are there for that. What policy reward is useful for, in this project, is comparing within the context family across iterations. For example, the collapse from approximately 25 on context_40m to approximately 0.95 on context_v2 is expected when fixing reward hacking.

Figure 4 plots Goalie/touch_rate_mean, which records the average per-step rate at which the tracked car records a ball touch. This is the most direct proxy for "is the goalie actually doing anything?" The dense run climbs steadily and ends with a touch rate above 0.045, which corresponds to the agent making contact with the ball with some regularity through the latter half of training. All three context runs hug near zero for the entire forty million steps, regardless of whether the run is the original hacked version or one of the corrected versions. The sparse run also sits at zero, which is consistent with its near-dead policy reward and KL diagnostics. The result for the context runs is the most surprising. The corrected context agents are not camping, as the positioning metrics in the next subsection show, but they are also not striking the ball in a way that the metrics logger picks up as a touch. This gap, between agents that are physically close to the ball and agents that are physically making contact with the ball, is the most important finding in the experiments.

Figures 5 and 6 plot the agent's positioning metrics: its mean distance to the ball, and its mean distance to its own goal line. Together, these metrics map exactly where each agent "lives"

on the field. In Figure 5 (Distance to Ball), the dense run starts far away at 4400 units and slowly migrates inward to end near 2900 units, showing a healthy evolution as the agent learns to engage. By contrast, the two corrected context runs immediately crash to roughly 2500 units and stay there, remaining the closest to the ball on average. Meanwhile, the original context_40m run sits higher at 3600 to 3800 units - indicating a car anchored near its net - while the sparse run sits in a similar lower range simply because it is wandering aimlessly. Figure 6 (Distance to Goal) reveals the inverse tracking data. The original context_40m run plummets to 1500 units from its own net and locks there, providing the "smoking gun" evidence for net camping. The two corrected context runs sit much higher, near 2600 units, proving the agent has stopped gluing itself to the goal line to patrol a wider region. Conversely, the dense run climbs to 7500 units before settling near 5700 units; because its reward only values moving the ball away from the net, so it roams incredibly far from its goal with no positional penalty. The sparse run remains flat at its kickoff starting distance of 4500 units.

The dense agent evolves into an aggressive ball-striker who frequently leaves the net exposed. The corrected context agents learn the opposite tradeoff - they maintain excellent positioning by staying close to both the ball and the net. However, as the touch-rate data shows, this defensive positioning does not translate into physical contact. Instead, they merely shadow the ball, tracking its movement at a small distance, rarely committing to a challenge.

Figure 7 shows that the dense run achieves a peak average normalized ball speed of approximately 0.11 and settles near 0.085, significantly outperforming all other agents by actively imparting force on the ball. In contrast, all three context runs remain near zero, and the sparse run displays a minimal 0.006 baseline from physics noise, showing a "contact gap" where context agents maintain close positioning but fail to move the ball. This highlights that the

context agents are content to occupy the same area as a near-stationary ball without taking action to change its state (because in hindsight that's safer than having the ball move anywhere).

Figure 8 plots Goalie/mean_boost_mean, showing that the original context_40m run hoards roughly 24% boost - three times higher than any other agent - due to its flawed +0.02 bonus for staying above a 20% threshold. The corrected context runs successfully collapse to between 8% and 10% boost on average, proving that dropping the weight to +0.005 eliminated the incentive to "boost farm." Meanwhile, the dense and sparse runs naturally drop below 5% boost because they lack any boost incentive. This transition provides the cleanest evidence that the v2 modifications broke the original exploit: the agent stopped hoarding boost because it stopped getting paid to do so, even if it failed to subsequently repurpose that resource into attacking the ball frequently.

Combining these findings, the profile of each agent becomes clear. The sparse agent learns nothing because its reward signal almost never fires during random play. The dense agent learns to actively strike the ball and make it move, accepting the tradeoff of frequently leaving its net exposed. The original context agent suffers from classic reward hacking; by prioritizing easy points for holding boost, facing the ball, and staying in net, it learns to camp in goal with a full tank while completely ignoring the ball. Finally, the two corrected context agents eliminate this exploit to become well-positioned "shadow" goalies; however, they merely follow the ball's movement at a distance without ever committing to a strike - a passivity that the v3 touch bonus marginally improved.

Discussion/Reflection

To keep it brief, the main points are: High reward does not equal good behavior. The highest-scoring agent (context_40m) was actually the most broken due to reward hacking. True performance must be evaluated using reward-agnostic behavioral metrics. Second, the corrected context agents became excellent at positioning but failed to strike the ball. This occurred because penalties made them risk-averse, and the sparse touch bonus (+0.30) wasn't enough to incentivize breaking their safe "shadowing" loop. Third, the simple, continuous PhysicsDenseReward outperformed the complex hybrid reward in ball interaction. Multi-term shaping rewards frequently trap agents in lazy, local optimums (like shadowing instead of striking). Fourth, none of these bots are human-competitive. At 40 million timesteps, this project isolates the mechanics of reward design under fixed constraints; it does not aim to compete with commercial bots trained on hundreds of billions of steps. Finally, to fix the contact gap, future iterations should abandon reward weight tuning and instead alter the environment. Some promising steps include launching the ball directly at the net during resets, training against frozen historical models to break passive self-play loops, or isolating mechanics via dedicated goalie drills.

Conclusion

This project compared three families of reward functions for training a PPO agent in a defensive leaning role in Rocket League, using RLGym v2 with RocketSim as the simulation backbone, rlgym-ppo as the PPO implementation, and Weights & Biases for built-in and custom metric logging. Five runs of forty million timesteps were trained under identical environment and PPO configurations, differing only in the reward function. The sparse run effectively failed to learn, which is the expected outcome when defensive-half touches are too rare under an untrained policy. The dense run produced the only agent whose behavior metrics clearly

demonstrate ball interaction: nonzero touch rate and ball speed, and a positioning curve that migrates from far from the ball to toward it over training, at the cost of being frequently roamed out of its own net. The original context-aware run was a classic case of reward hacking, with a very high policy reward driven entirely by passive shaping terms while the car camped near the goal line with a near-full boost meter. Context v2 successfully suppressed the exploit through reduced passive weights and explicit no-touch and center-unchallenged penalties, lowering the boost level from approximately 24 percent to under 10 percent and raising distance to own goal from approximately 1500 to approximately 2600 units, but it did not recover dense-like contact: the agent shadowed the ball, rarely committing a strike. Context v3 added an event-based touch bonus and produced slightly higher policy reward and slightly more stable PPO updates than v2, but touch rate and ball speed remained low near zero, confirming that scalar tuning alone was not sufficient to close the contact gap but minor improvements were seen.

The practical lesson is that reward design matters more than reward magnitude. A high scalar policy reward can hide a degenerate policy when the formula is misaligned with the actual task, and the most reliable cross-strategy diagnostic is a small suite of behavior metrics that reflect what the agent should be doing in the environment. The results also show that not every reward problem can be solved by adding more reward terms; once an agent settles into a risk-averse local optimum, escaping it generally requires changes to the state distribution it trains on rather than additional shaping. Future work would most usefully address that distribution directly, either through small variations to the kickoff-only setup or through a dedicated goalie-drill setup that decouples positioning from striking. The code, checkpoints, and W&B artifacts produced during this project are sufficient to support those next steps without retraining or invalidating any of the comparisons reported here, which tell its own important story.

References

- AechPro. (n.d.). *AechPro/rlgym-PPO: A vectorized implementation of PPO for use with RLGym*. GitHub. <https://github.com/AechPro/rlgym-ppo>
- RLGym: RLGym. RLGym RSS. (n.d.). <https://rlgym.org/>
- Rolv-Arild. (n.d.). *Rolv-Arild/Necto*. GitHub. <https://github.com/Rolv-Arild/Necto>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, August 28). *Proximal policy optimization algorithms*. arXiv.org. <https://arxiv.org/abs/1707.06347>
- VirxEC. (n.d.). *VIRXEC/RLVISER: A visualizer for the rocketsim project*. GitHub. <https://github.com/VirxEC/rlviser>
- ZealanL. (n.d.). *ZEALANL/RocketSim: A C++ library for simulating Rocket League games at maximum efficiency*. GitHub. <https://github.com/ZealanL/RocketSim>

Appendix A: Figures

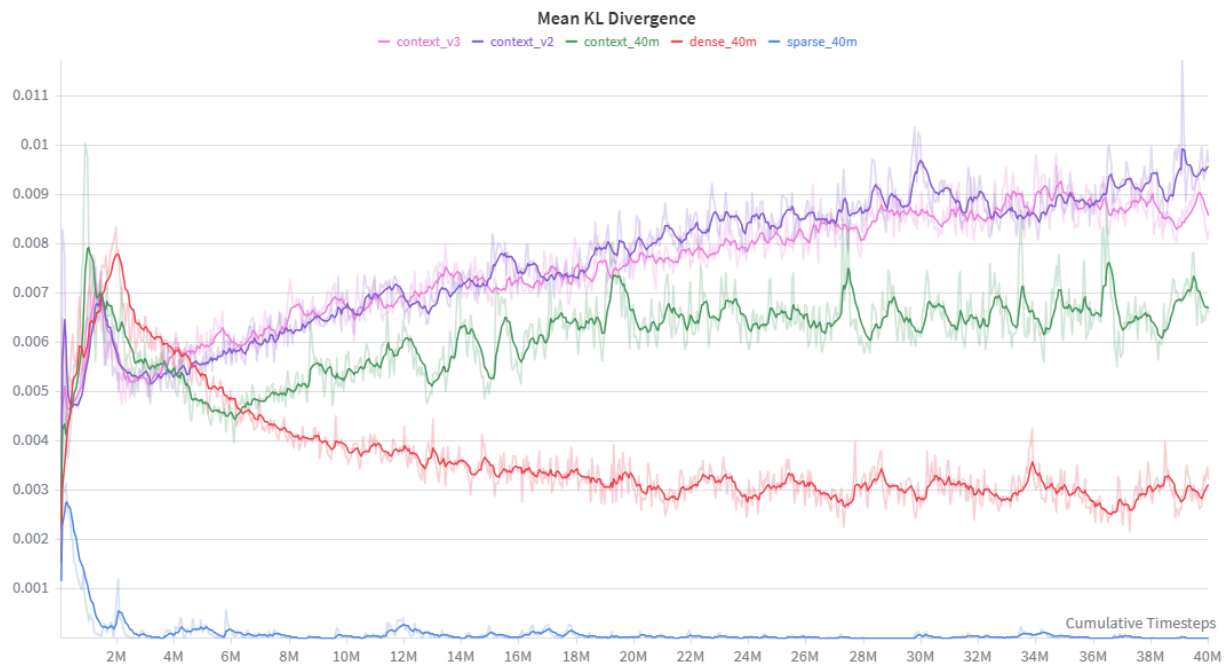


Figure 1: Mean KL divergence over training (sparse_40m, dense_40m, context_40m, context_v2, context_v3)

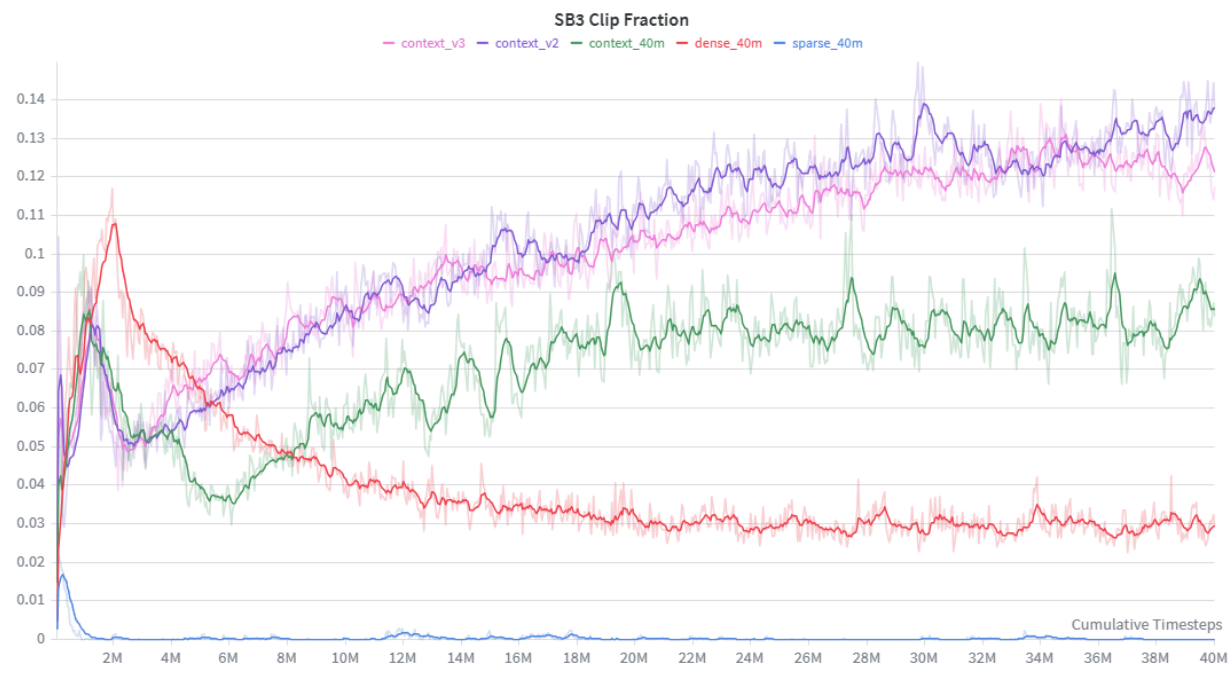


Figure 2: SB3 clip fraction over training (same five runs)



Figure 3: Policy reward over training (same five runs)

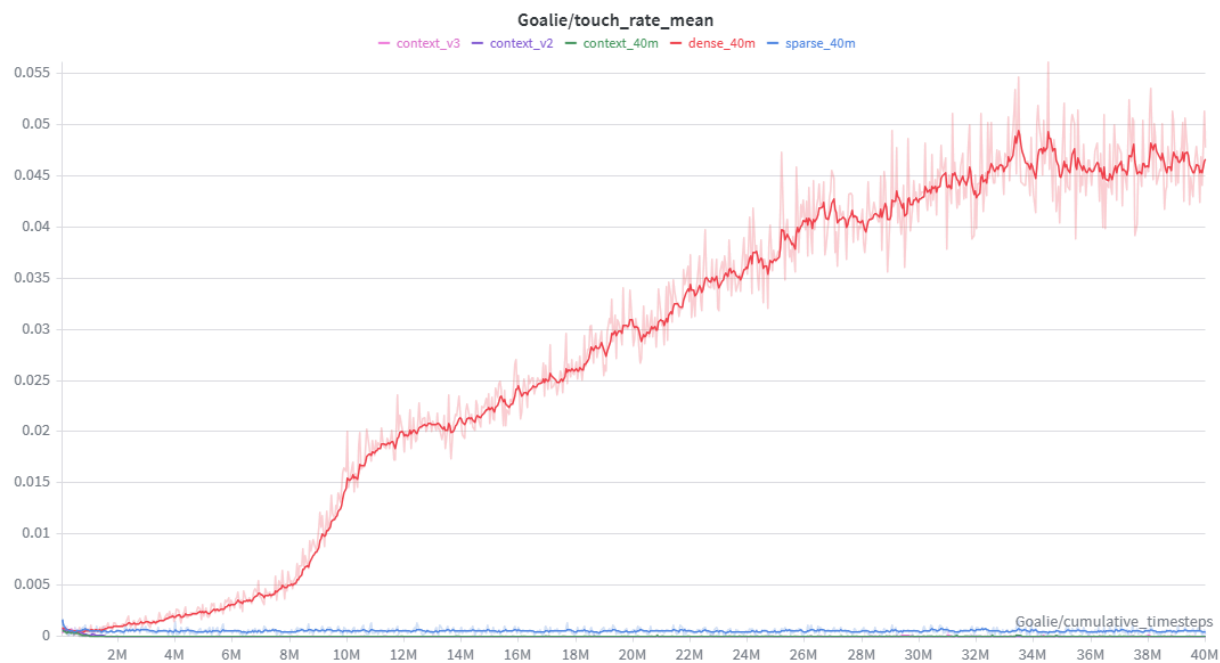


Figure 4: Goalie/touch_rate_mean over training

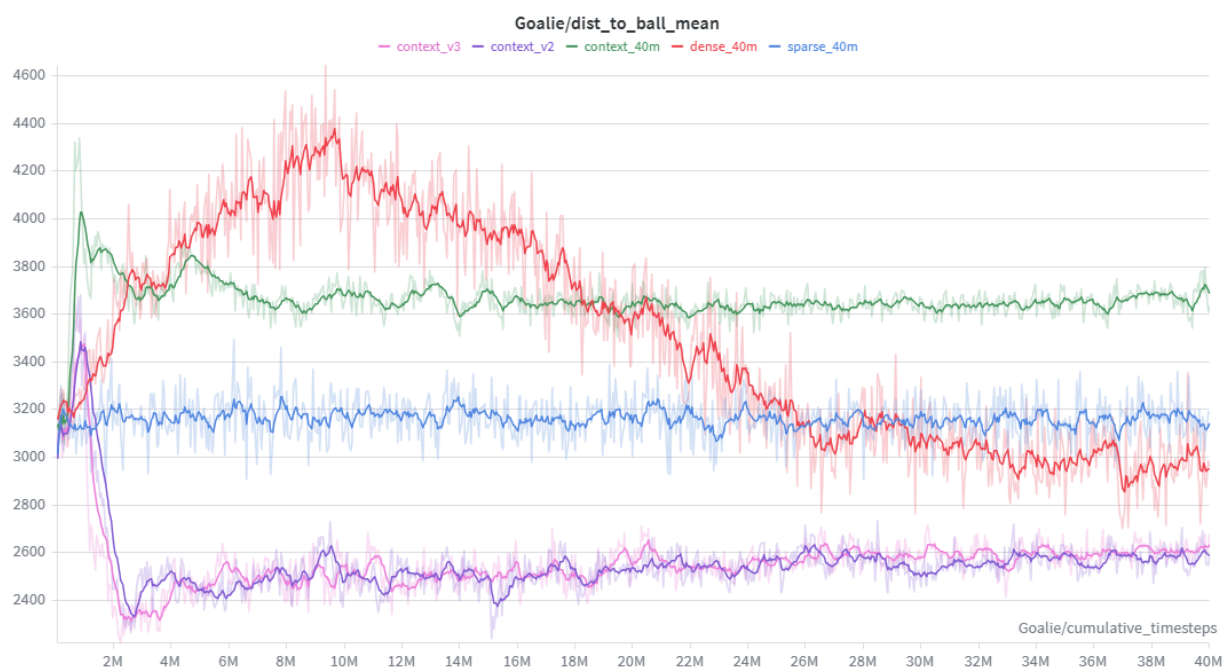


Figure 5: Goalie/dist_to_ball_mean over training

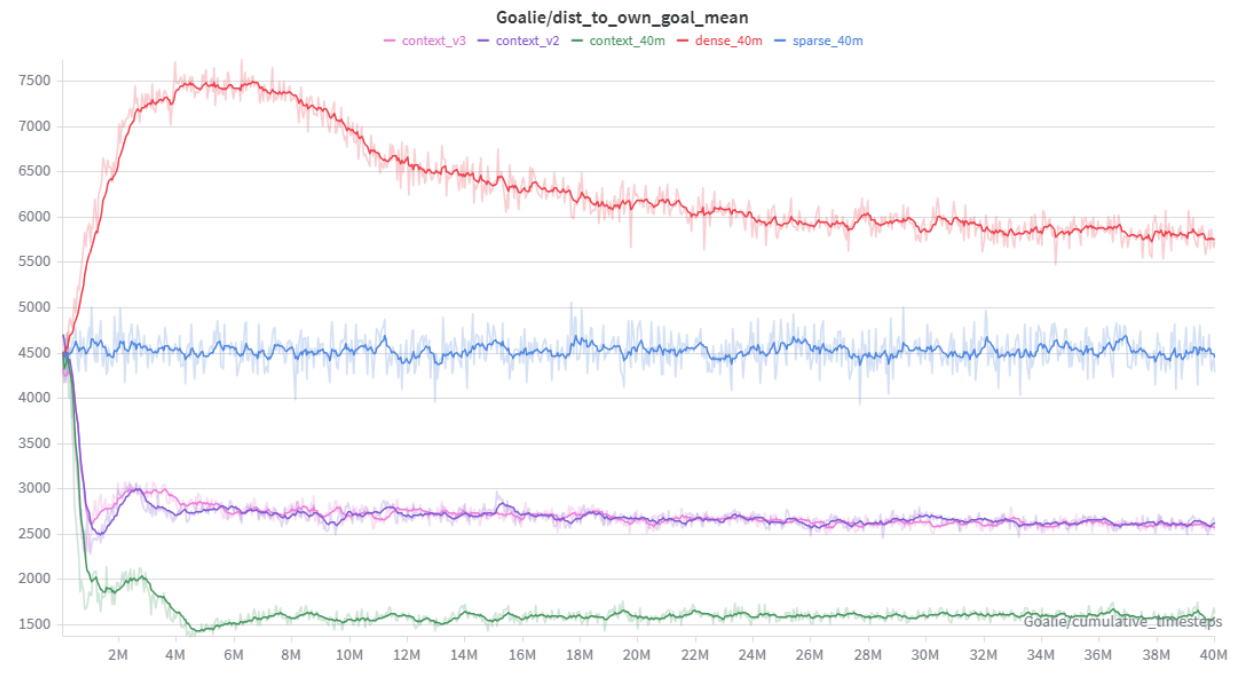


Figure 6: Goalie/dist_to_own_goal_mean over training

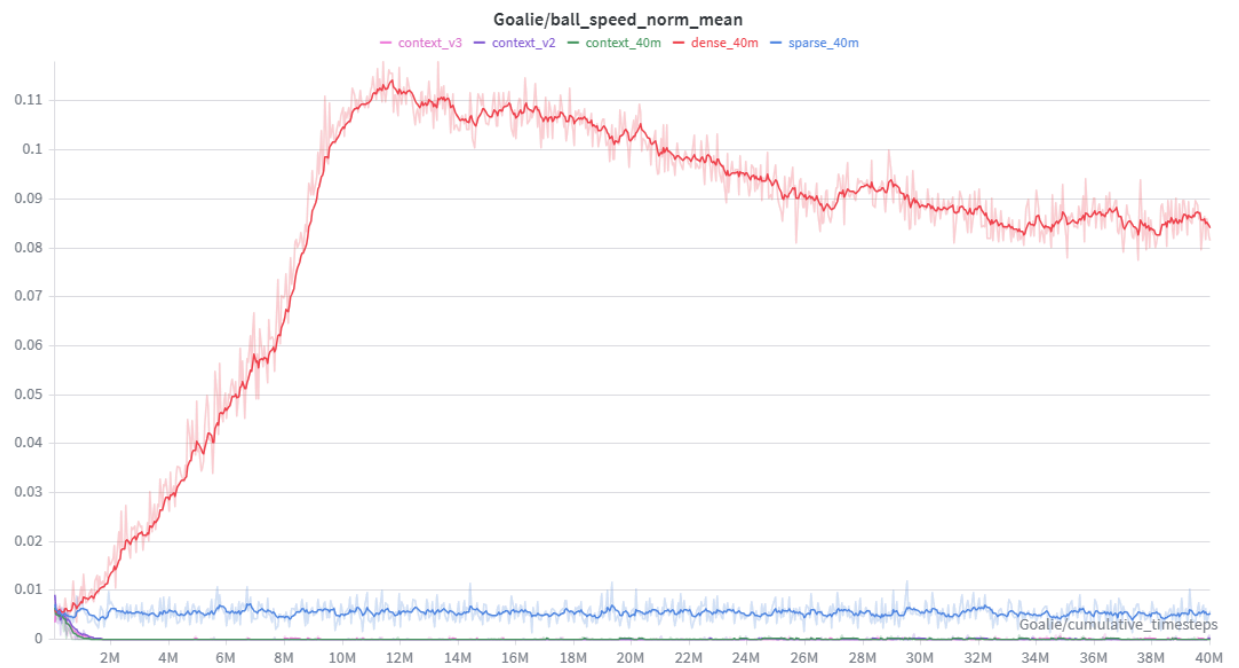


Figure 7: Goalie/ball_speed_norm_mean over training

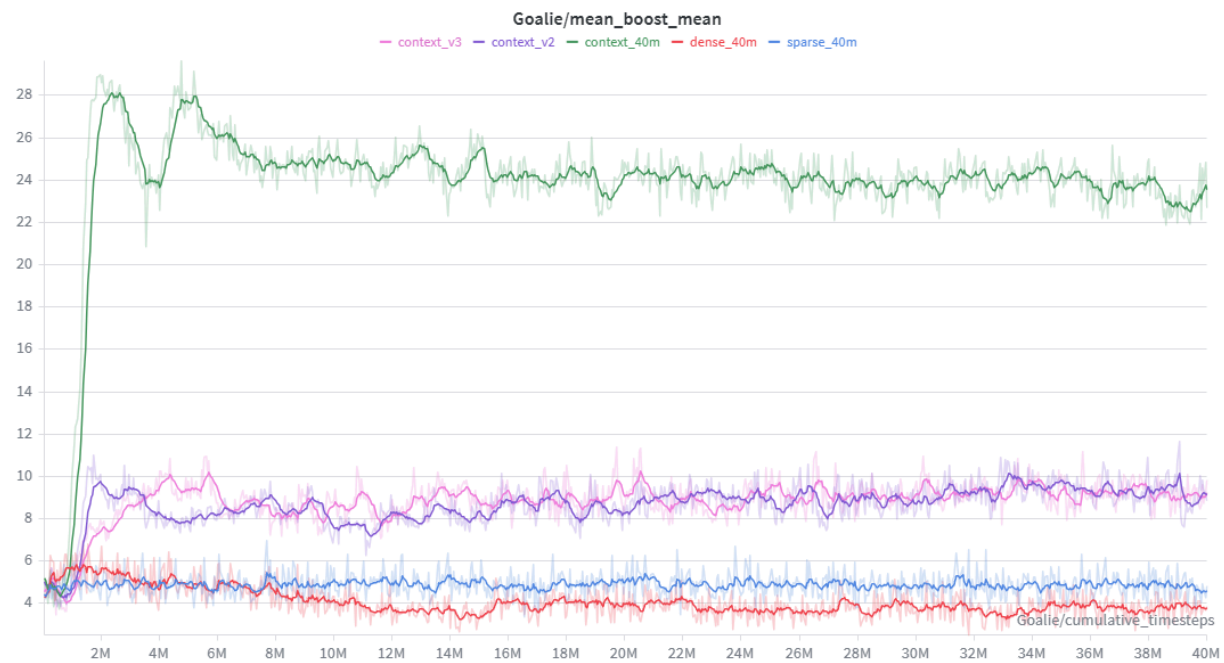


Figure 8: Goalie/mean_boost_mean over training

Appendix B: Videos

Sparse_40m:

<https://drive.google.com/file/d/1eYVl0I2iisNpYHPvyfPsBx8LpehN3P2t/view?usp=sharing>

Dense_40m:

<https://drive.google.com/file/d/1ZQlz9Vw4eBNNCMKKI-vKwDNA-GQ46lTA/view?usp=sharing>

Context_40m:

<https://drive.google.com/file/d/15dLHT5Ilw1QM5xDYNnH29SG7xIMWGB0/view?usp=sharing>

Context_v2:

<https://drive.google.com/file/d/1dCuEvIvFEb1LMDZQiGj6DjJRMKydyFIO/view?usp=sharing>

Context_v3:

https://drive.google.com/file/d/1fWNQtuZb-rYYGxlsYWN_4OnvjH7lIQZd/view?usp=sharing